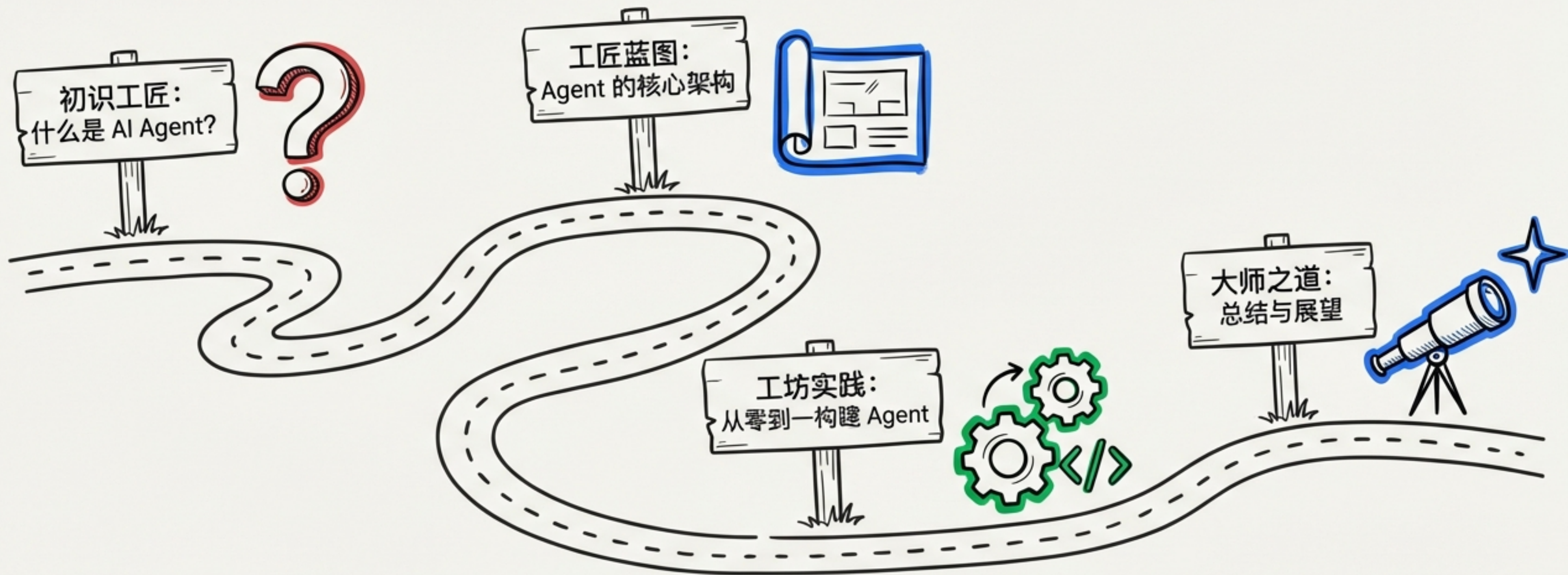


Agents 最佳实践指南

打造你的第一个智能“数字工匠”

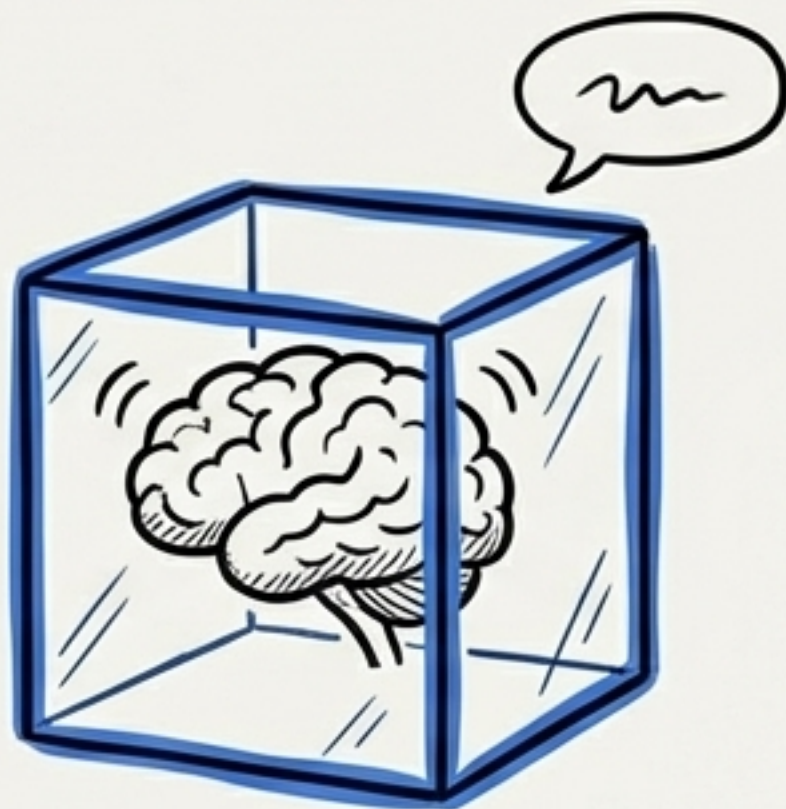


本次探索之旅



从“聊天机器人”到“行动派”

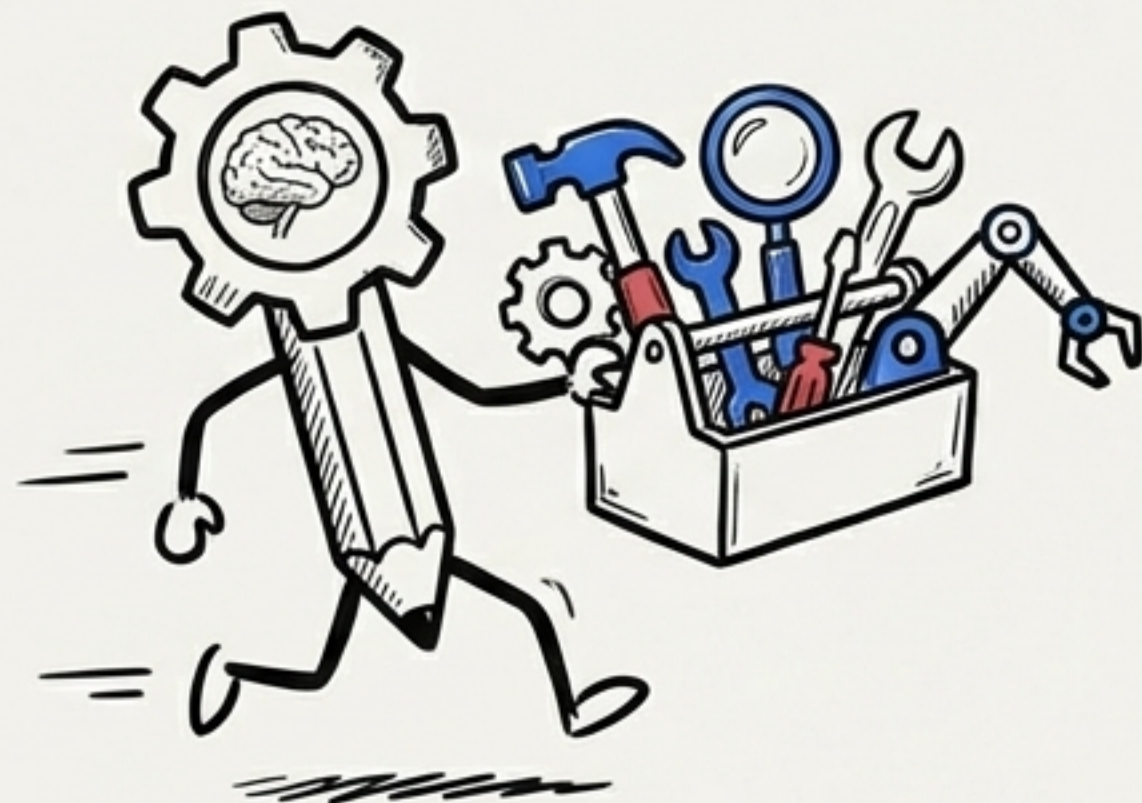
传统LLM



进化



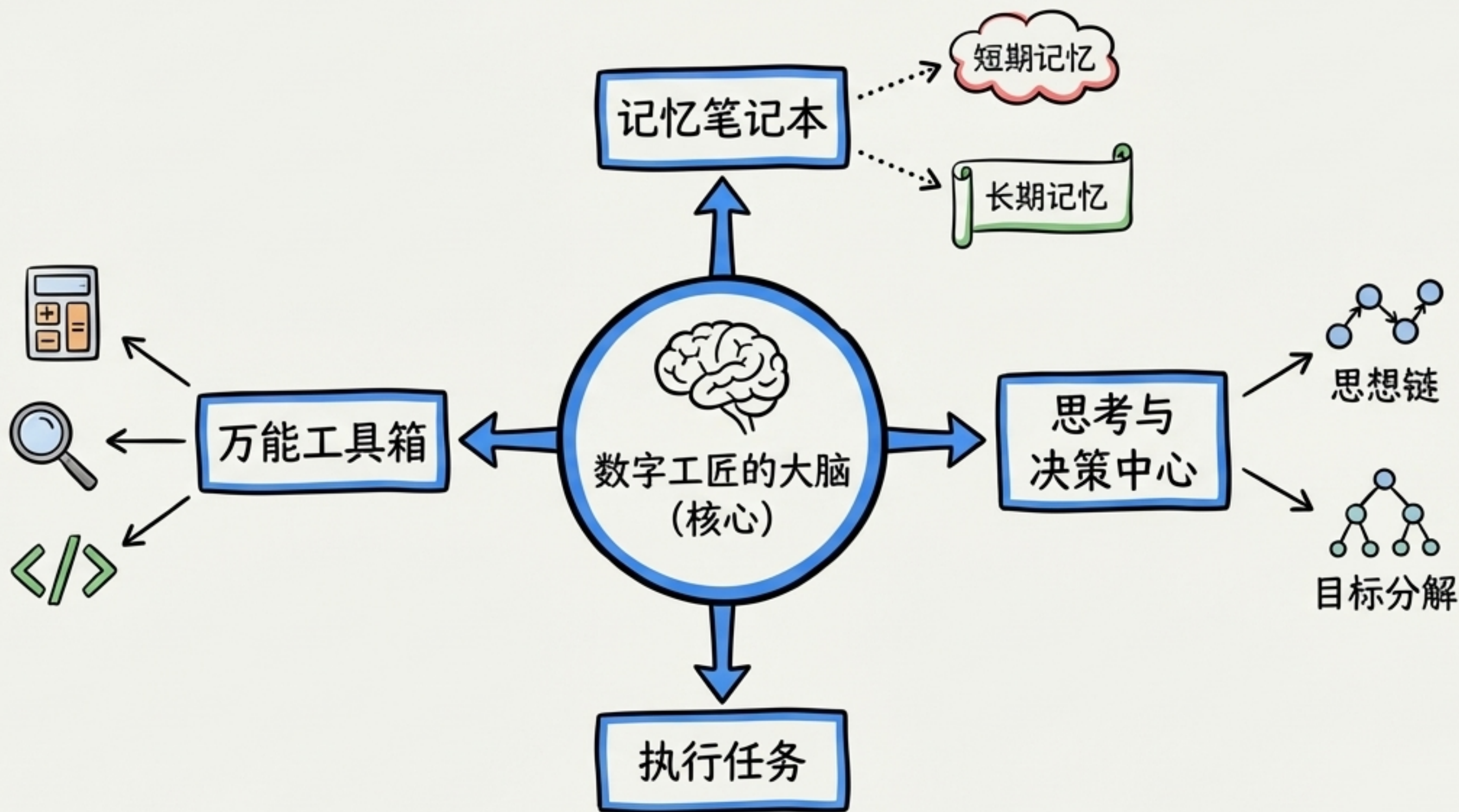
AI Agent



- 擅长对话、生成内容
- 但无法与外部世界交互、执行任务

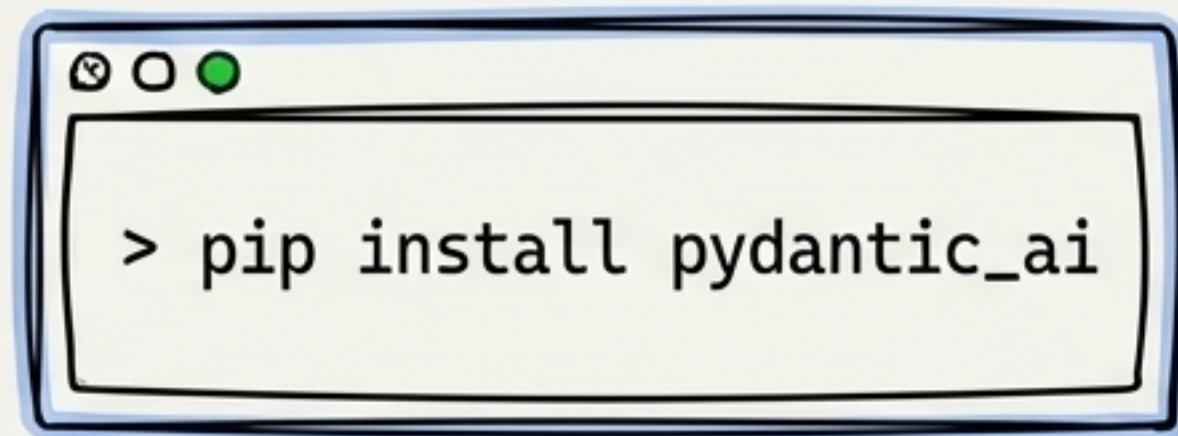
- 不仅能思考，更能行动
- 使用工具、访问数据、完成复杂目标

数字工匠的“解剖学”



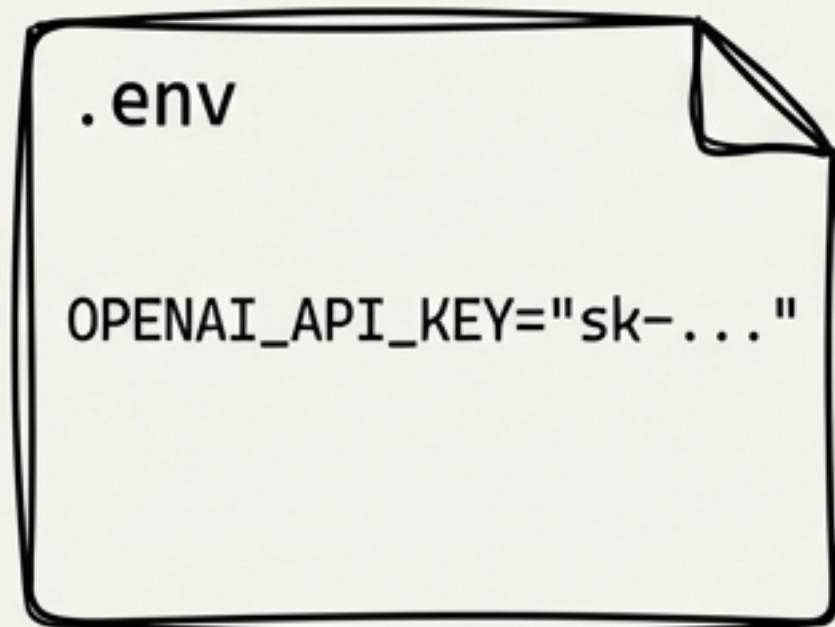
第一步：准备工坊

1. 安装必要的库



```
> pip install pydantic_ai
```

2. 管理你的“钥匙”



.env

OPENAI_API_KEY="sk-..."

将密钥与代码分离，保障安全



第二步：挑选大脑并导入零件

导入所需的“零件”

```
from pydantic_ai import Agent
from pydantic_ai.models import OpenAIModel
import tools
from dotenv import load_dotenv
```

给工坊通电，加载环境变量
load_dotenv()

选择一个强大的“大脑” (GPT-4o)
model = OpenAIModel('gpt-4o')

工匠本体

大脑型号

我们为工匠准备的工具箱

选择 GPT-4o 作为思考核心

第三步：赋予角色与技能

```
# 实例化一个 Agent  
agent = Agent(  
    model=model,
```

设定角色和使命
(这是工匠的行为准则和灵魂!)

```
    system_prompt="你是一名经验丰富的程序员",
```

```
    tools=[  
        tools.read_file,  
        tools.list_files,  
        tools.rename_file
```

把工具箱交给他
(他现在会用这些工具了!)

```
    ]
```

```
)
```

装上我们选好的大脑

第四步：启动工作循环并植入记忆

```
def main():  
    history = []
```



为工匠准备一个空的
“记忆笔记本”

```
    while True:
```

```
        user_input = input("Input: ")
```

让工匠带着“笔记本”开
始工作，以便参考上下文

```
        resp = agent.run_sync(user_input, history=history)
```

```
        history = list(resp.all_messages)
```



```
        print(resp.output)
```

打印出工匠的成果

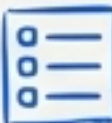
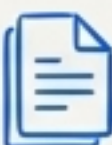
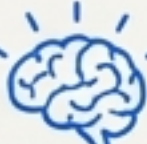
将新的对话和行动
记录到“笔记本”中

旁注：没有`history`，每次对话都是一次遗忘。记忆是持续性任务的关键。

实践成果：看，我们的工匠在工作！

➤ Input: 查询当前文件，告诉我'b'文件用的是什麼语言，功能是什麼？

Agent思考过程

-  *用户要查询文件... 调用 `list\_files` 工具。
-  *用户想知道 'b' 文件的内容... 调用 `read\_file` 工具读取 'b'。
-  *分析文件内容... 这是我的强项。

➤ 文件 'b' 的内容是 `'print("Hello, World!")'`。这是一个 Python 脚本，用于在控制台打印字符串。

揭秘：工匠的思考过程

> User: list files

[Thinking...]

The user wants to see the files.
I should use the 'list_files' tool.

[Tool Call: list_files()]

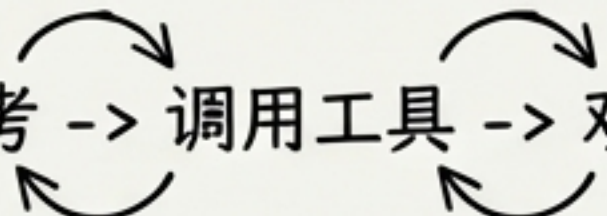
[Tool Output: a, b, c, d, e, f]

[Thinking...]

I have the file list. Now I will present it to the user.

> Agent: The files are a, b, c, d, e, f.

Agent 的每一步行动都基于“思考 -> 调用工具 -> 观察结果”的循环。



成为优秀“工匠建筑师”的准则



🎯 明确的指令：

System Prompt 要清晰、具体。它定义了 Agent 的角色、能力边界和行事风格。



🔧 精良的工具：

工具的描述（docstring）至关重要。这是 Agent 理解工具用途的唯一依据。描述越清晰，调用越准确。



🧠 选择合适的大脑：

不同的任务需要不同能力的 LLM。在成本和性能之间找到平衡。



📝 善用记忆：

对于需要上下文的连续任务，设计有效的记忆管理机制是成功的关键。

工匠的世界：不必重复造轮子



Pydantic AI（我们今天使用的）：轻量、简洁，与 Pydantic 结合紧密，适合快速开发。



LangChain：功能最全面、社区庞大，是行业标准之一。



LlamaIndex：专注于 RAG（检索增强生成），构建知识型 Agent 的利器。



AutoGen (Microsoft)：擅长构建多个 Agent 协同工作的复杂系统。

选择合适的框架，就像为工匠选择合适的工坊。

未来的“工匠”：从个体到协作工坊

1. 多 Agent 系统[💡]: 多个 Agent 协同工作，分别扮演规划师、执行者、审查员等不同角色，解决更复杂的问题。
2. 自我进化[📈]: Agent 根据任务结果自我反思，自动优化其提示词或工具使用策略。
3. 连接物理世界[🔍]: Agent 不仅能操作软件 API，还能控制机器人、无人机等硬件设备。



核心要点回顾

- Agents 的核心价值：将 LLM 的“思考”能力转化为可与外部世界交互的“行动”能力。
- Agent 的三大支柱：大脑（LLM）负责推理，工具（Tools）负责执行，记忆（Memory）负责联系上下文。
- 成功的关键：成功的 Agent = 明确的指令（Prompt）+ 精心设计的工具（Well-designed Tools）。

开启你自己的创造之旅

Next Steps

推荐探索：

- `pydantic-ai`官方文档 - 发现更多模型和高级功能。

挑战任务：

- ☐ 为你的 Agent 添加一个“网络搜索”工具。
- ☐ 尝试更换一个不同的 LLM（如 Google Gemini）。
- ☐ 让你的 Agent 能够创建和写入新文件。